CLAUDE SKILLS SECURITY

# Attack Path Diagrams

Claude Skills Injection —
Threat & Defense Guide

VERSION 1.0.0

DOCUMENT attack-path-diagrams

RationalEyes.ai

contact@rationaleyes.ai

March 2026

# Table of Contents

Diagram 1: Attack Path Overview

Diagram 2: Defense Architecture

Diagram 3: Promptware Kill Chain Mapping

Diagram 4: Skill Architecture — Load and Execution Flow

Diagram 5: Multi-Agent Propagation

Diagram 6: Risk Matrix — All 12 Vectors

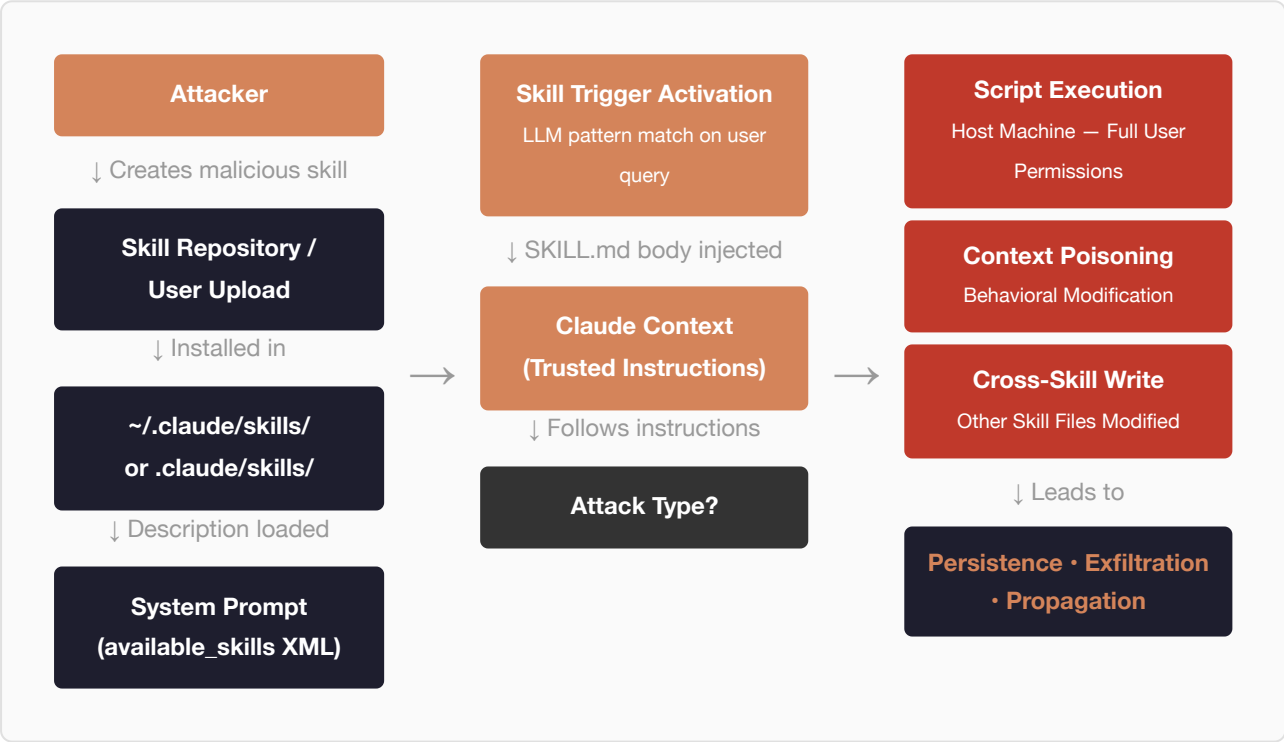
Diagram 7: Defense Control Mapping

Notes

> ***Purpose:*** *Visual representations of the Claude Skills injection attack surface, defense architecture, and threat propagation patterns. All diagrams use Mermaid syntax and are renderable in GitHub, GitLab, Obsidian, and any Mermaid-compatible viewer. This PDF-ready version includes both rendered visual representations and Mermaid source for re-rendering.*

# Diagram 1: Attack Path Overview

How a malicious skill moves from creation to full host compromise.




*Attack path from skill creation through trigger activation to impact*

## Mermaid Source

```
flowchart TD
    A[Attacker] -->|Creates malicious skill| B[Skill Repository / User Upload]
    B -->|Installed in| C[~/.claude/skills/ or .claude/skills/]
    C -->|Description loaded at startup| D[System Prompt - available_skills XML]
    D -->|LLM pattern match on user query| E[Skill Trigger Activation]
    E -->|SKILL.md body injected| F[Claude Context - Trusted Instructions]
    F -->|Follows instructions| G{Attack Type}
    G -->|Script execution| H[Host Machine - Full User Permissions]
    G -->|Context poisoning| I[Behavioral Modification]
```

```
    G -->|Cross-skill write| J[Other Skill Files Modified]
    H -->|Output returns to| F
    H -->|Persistence| K[Cron Jobs / New Skills / Modified Configs]
    H -->|Exfiltration| L[External C2 Server]
    I -->|Affects subsequent| M[All Future Responses]
    J -->|Propagates to| N[Other Agents in Team]
```

# Diagram 2: Defense Architecture

Five-layer defense-in-depth model for skill injection protection.

**Layer 5: Monitoring** — OpenTelemetry → Behavioral Analysis → SIEM/Audit Logs (Detection + Response)

**Layer 4: Sanitization** — Output Scanning → Credential Redaction → Injection Detection (Detection)

**Layer 3: Sandbox** — Seccomp-BPF → Namespace Isolation → Env Filtering (Containment)

**Layer 2: Scope** — Trigger Allowlist → Permission Defaults (Prevention)

**Layer 1: Integrity** — Signing → Hash Verification → Static Analysis (Prevention)

*Defense layers from prevention (bottom) through containment and detection to response (top)*

## Mermaid Source

```
flowchart LR
    subgraph L1[Layer 1: Integrity]
        S1[Signing] --> S2[Hash Verification]
        S2 --> S3[Static Analysis]
    end
    subgraph L2[Layer 2: Scope]
        T1[Trigger Allowlist] --> T2[Permission Defaults]
    end
    subgraph L3[Layer 3: Sandbox]
        U1[Seccomp-BPF] --> U2[Namespace Isolation]
        U2 --> U3[Env Filtering]
    end
    subgraph L4[Layer 4: Sanitization]
        V1[Output Scanning] --> V2[Credential Redaction]
        V2 --> V3[Injection Detection]
    end
    subgraph L5[Layer 5: Monitoring]
        W1[OpenTelemetry] --> W2[Behavioral Analysis]
        W2 --> W3[SIEM/Audit Logs]
    end
    L1 --> L2 --> L3 --> L4 --> L5
```

# Diagram 3: Promptware Kill Chain Mapping

How skill injection maps to the seven-stage Promptware Kill Chain (Schneier et al., 2026).

*Promptware Kill Chain — skill injection natively covers all seven stages*

# Diagram 4: Skill Architecture — Load and Execution Flow

How skills are discovered, loaded, and executed within Claude Code.

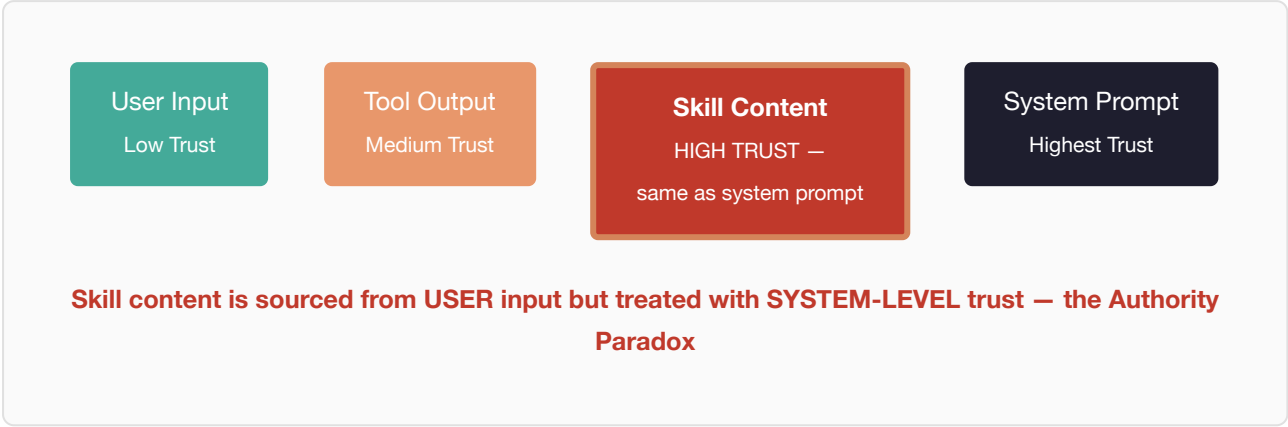
## Discovery Phase

| PRIORITY | SOURCE | SCOPE |
|---|---|---|
| 1 (Highest) | Enterprise Managed Settings | Organization-wide |
| 2 | Personal Skills (~/.claude/skills/) | User-wide |
| 3 | Project Skills (.claude/skills/) | Project-scoped |
| 4 | Plugin-Provided Skills | Plugin-scoped |
| 5 (Lowest) | Bundled Skills | Anthropic-provided |

## Loading Phase

| STEP | ACTION | SECURITY RELEVANCE |
|---|---|---|
| 1 | YAML Frontmatter parsed (name, description, allowed-tools, hooks) | Self-declared permissions |
| 2 | Descriptions injected into system prompt as `available_skills` XML | Controls trigger matching |
| 3 | Full SKILL.md body loaded on trigger as `isMeta` user message | System-level trust for user content |
| 4 | scripts/ executed via Bash tool on demand | Full host permissions |
| 5 | references/ loaded into context on demand | Large context injection surface |

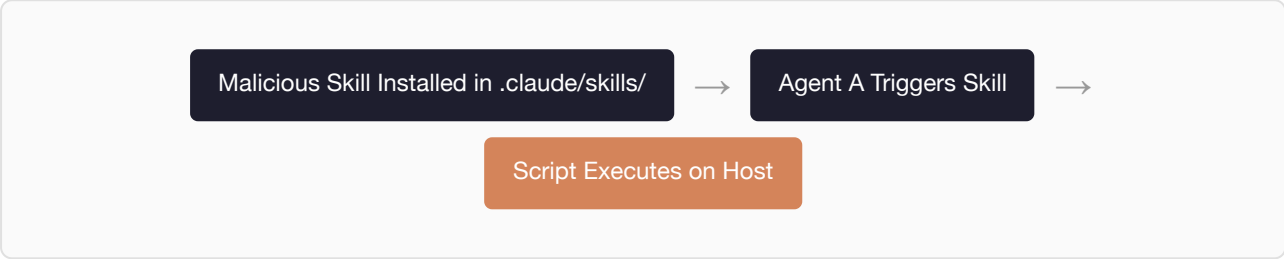
## Trust Model — The Vulnerability



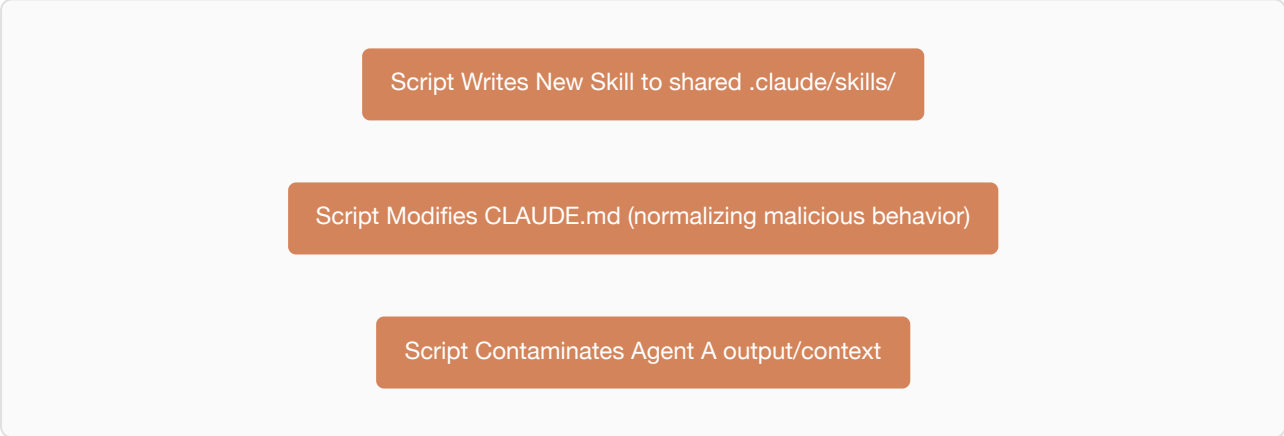
# Diagram 5: Multi-Agent Propagation

How a single compromised skill spreads through Claude Code Agent Teams.

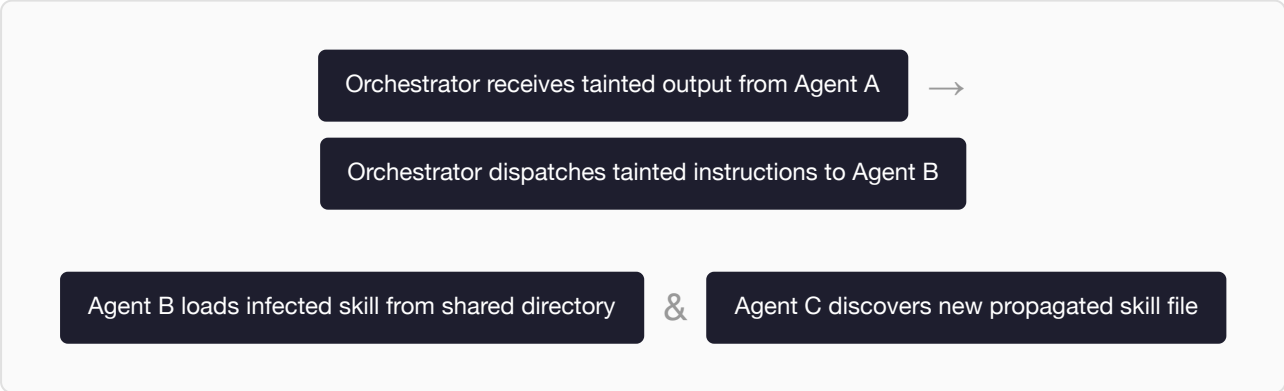
## Stage 1: Initial Infection




## Stage 2: Propagation

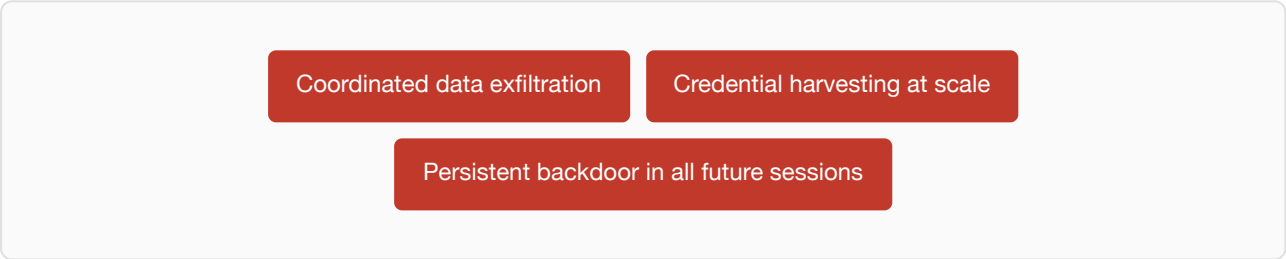



## Stage 3: Agent Team Spread



## Stage 4: Persistence

## Stage 5: Actions on Objective



# Diagram 6: Risk Matrix — All 12 Vectors

Visual risk assessment for all taxonomy vectors. Axes: Attack Complexity (horizontal) vs. Risk Rating (vertical).

| QUADRANT | VECTORS | CHARACTERIZATION |
|---|---|---|
| **High Priority — Easy to Deploy**<br>Low Complexity, High Risk | SKI-001 Content Poisoning<br>SKI-003 Persistence<br>SKI-012 Auth Paradox<br>SKI-007 Metadata Manipulation<br>SKI-002 Trigger Hijack | These vectors require minimal attacker sophistication but pose the highest immediate risk. They are the first vectors an adversary would exploit. |
| **Critical Priority**<br>High Complexity, High Risk | SKI-009 Multi-Agent Propagation<br>SKI-005 Context Poisoning<br>SKI-010 Cache Poisoning | Sophisticated attacks with the highest impact potential. Require multi-stage execution but produce the most severe outcomes. |
| **High Priority — Moderate**<br>Medium Complexity, High Risk | SKI-008 Supply Chain<br>SKI-011 Skill-as-C2<br>SKI-004 Script Compromise | Real-world exploitation confirmed (ClawHavoc). Represent the current active threat landscape. |
| **Monitor**<br>High Complexity, Lower Risk | SKI-006 Skill Chaining | Requires multiple installed skills and careful timing. Lower probability but meaningful impact when achieved. |

# Diagram 7: Defense Control Mapping

Which defense layers address which attack vectors.

| DEFENSE CONTROL | VECTORS ADDRESSED |
|---|---|
| **L1: Cryptographic Signing + Hash Pinning** | SKI-003 (Persistence), SKI-006 (Supply Chain) |
| **L1: Static Analysis at Install Time** | SKI-001 (Content Poisoning), SKI-003 (Persistence), SKI-006 (Supply Chain) |
| **L2: Trigger Scope Allowlist** | SKI-002 (Trigger Hijacking) |
| **L2: context:fork Mandatory Isolation** | SKI-009 (Multi-Agent) |
| **L3: Script Sandboxing (Seccomp + Namespaces)** | SKI-004 (Script Compromise) |
| **L3: Env Var Filtering (No secrets in skill scope)** | SKI-004 (Script Compromise), SKI-011 (Skill C2) |
| **L4: Output Sanitization (CaMeL-style)** | SKI-001 (Content Poisoning), SKI-005 (Context Poisoning) |
| **L5: Behavioral Monitoring (OpenTelemetry + SIEM)** | SKI-005 (Context Poisoning), SKI-009 (Multi-Agent), SKI-011 (Skill C2) |
| **Architecture: Trust Differentiation by Provenance** | SKI-012 (Authority Paradox) |

# Notes

- All diagrams are for security research and educational purposes.
- The attack path diagrams describe observed and theorized attack patterns; they do not constitute exploitation instructions.
- Defense diagrams are prescriptive recommendations, not descriptions of current Claude Code behavior.

- Mermaid source code is included with each diagram for re-rendering in compatible viewers. Use mermaid.live for interactive rendering.
- Mermaid `quadrantChart` requires Mermaid v10.3+.